

using scanf allows a variable to be called

identifiers in can start of with a letter from A-Z or an \_ symbol. there can't be numbers in a variable name.

EOF = End of File, is commonly -1.

-----Remember-----

int = integers  
using scanf("%d", &name) *this tells scanf+printf how to format the data.*  
allows the %d(decimal integer) and the &name(variable name) to read a decimal values stored in the variable.

While loop

Is used to repeat a condition as long as it remain true.

While loop doesn't need a ;  
do while loop  
allows a condition to start and continue if true.

-----continued function -----

Mainly allows a condition to stay in a loop but jump to the next round of the expression.

within a while loop,  
it will jump to the top and test the expressions.

Do while will make the condition jump to the bottom.

for loops will make the condition jump to expression3 and retest expression2.

-if a condition expression is true;  
otherwise it is false

standard input -- keyboard  
standard output -- video

%variable <- data.in  
%variable >data.out

Function calls

will have an associated overhead.  
works in a system that must copy any arguments that pass through the function, which will track of where to resume the program when the function returns.  
No type of checking is needed in 0a macros.

It have an advantage of using the min function to find the minimum of any two numerical data type.  
But alsoa has the disadvantage of introducing errors that can be difficult to uncover.  
like producing presumable useless values  
when the argument is a string and the other is a float.

---File-----

Old way  
scanf()  
printf()

New way

fin= is a pointer to file input  
fout = is a pointer to file output.  
fopen will allow you to open the file for reading or writing.  
fin with fopen = allows reading only  
fout with fopen = allows writing only.  
Example:  
fopen("filename", "r") <- means reading)  
"w" <- means writing'  
"variable name"

fscanf is used to read the formatted data from a file.  
int fscanf(filename"stream, const char\*format,...);  
fprintf is used to write formatted data to a file.  
int fprintf(filename"stream,const char \*format, ...);

-----If statement-----

is a function statement  
that allows a condition to be checked once.  
if else =function statement is used when you condition needs to be checked by different expressions.

the difference between while and if loops  
while will continue to check until the condition runs true.  
if loops will only check once.

-----break-----

Break allows the condition to exit from the innermost loop  
while, do while,for, and switch loop statements are the only ones that can allow break to function properly.

---- Switch statement-----

switch loop function allow a condition to be tested like a multiple if else if statment.  
Note that if the condition doesn't satisfy any of the cases, the condition will go through a default case instead.  
P.S each case must have a break function at the end of the code  
to avoid causing an infitite loop.

OR → ||  
AND → &&  
NOT → !

---cast operator---

the cast operator will convert one data type to another.  
like an int to a float.  
te way to bring a cast into a line of code is (float) or (data type).

-----getchar and putchar-----

the way it will and can be written is:  
variable = getchar(); this allows a single character to be assigned to a variable.  
putchar(variable); this allows the character, that is being stored, to be outputed in standard form.

-----Macro-----

these are usually the directives like #include, #define,etc.  
which allows the to do specific functions.  
it can also perform simple substitution.

parameterized macro  
it can allow for code to be reused and customizable at the preprocessor stage befor it compiles.

Decleration - will tell the compiler that something exists somewhere.  
but doesn't allocate memory and can be declared multiple times.  
int add(int a, int b);

Definition - creates an object, which will allocate memory for the object.  
can only be delcared once and can be used in the declaration.  
int Var = #;

A header file will allow you to use declarations types to allow multiple source files to be use without implementation. It can also allow you to reuse code across multiple files.  
This file type allows for functions to do interact , while the source file contains the main implementation.  
It can also reduce the mistakes of duplicate headers.This acts like a table of contents for code.

System header - #include <stdio.h> // will have angle brackets.  
Local header - #include "my\_header.h" // Will use quotes.

extern is used to declare a variable/ function without needing to define it. It lets the compiler know that something exists. similar to the declaration.  
----- Misc. Notes -----  
#line <- will specifies line number by compiler.

The #pragma directives offer a way for each compiler to offer machine- and operating-system-specific features while retaining overall compatibility with the C and C++ languages. Pragmas are machine- or operating-system-specific by definition, and are usually different for every compiler.  
#pragma token-string  
inline- insert a copy of the function when it is called

< ← less than  
> ← greater than  
≤ ← less than or equal to  
≥ ← greater than or equal to  
!= ← not equal.

*Array Notes*

*finding the largest element in an array*  
int largest(int arr[], int size){  
if(size <= 0) {  
printf("Error: Array is empty\n");  
return -1; // this returns an error value.  
}  
int largestNum = arr[0];  
  
for(int i = 1; i < size; i++){  
if(arr[i] > largestNum){  
largestNum = arr[i];  
}  
return largestNum;  
}

Array notes  
declaration  
int a[number value];  
just know that you cant go under or over the bounds of the number value.  
group initialization  
int c[] = {#, #, #, #, #}  
printf("%d\n", c[#]) - will print out the number value in that address space.

## Sample code

### increment definition

++i (pre increment) - is the first increment, meaning it will add a new value to a list.

// Pre-increment

```
printf("++i: %d\n", ++i); // Output: 6 (incremented first)
```

```
printf("After ++i: i = %d\n", i); // i is now 6
```

++ (post increment) - allows the current value to increment to itself.

// Post-increment

```
printf("j++: %d\n", j++); // Output: 5 (original value used first)
```

```
printf("After j++: j = %d\n", j); // j is now 6
```

### getting the maximum number

```
#include <stdio.h>
```

int max\_three(int a,int b,int c) // This allows us to define our variables.

if (a>=b && a>=c) // This nested if else statement will check the var. to the conditions.

```
return a;
```

```
} else if(b>=a && b>=c){
```

```
return b;
```

```
} else {
```

```
return c;
```

```
}
```

```
}
```

int main(){ // This function allows us to get input and print out the max.

```
int one, two, three;
```

```
printf("Enter three integers: ");
```

```
scanf("%d %d %d", &one, &two, &three);
```

```
int max = max_three(one, two, three);
```

```
printf("The maximum of the integers is: %d\n", max);
```

```
return 0;
```

```
}
```

Parameterized Macros compared to Functions

No arguments are passed to a parameterized macro (min).

parameterized macros returns no values.

preprocessor merely replaces the macros with the definition.

So min( num1, num2);

by

```
((num1) < (num2)) ? (num1) : (num2);
```

^ this is used to be more efficient than a function.

### prime example

```
#include <stdbool.h>
```

// this is used from a previous assignment but just modify at the main function.

```
bool prime(int num){
```

```
if (num <=1){
```

```
return false;
```

```
}
```

```
if (num == 2){
```

```
return true;
```

```
}
```

```
if(num %2 == 0){
```

```
return false;
```

```
}
```

```
for(int i = 3; i * i <= num; i+=2){
```

```
if (num %i ==0){
```

```
return false;
```

```
}
```

```
}
```

```
return true;
```

```
}
```

```
int main(){
```

```
int n;
```

```
printf("Enter a number: ");
```

```
scanf("%d", &n);
```

```
if(prime(n)){
```

```
printf("%d : This is a prime. \n", n);
```

```
} else {
```

```
printf("%d : This is not a prime. \n", n);
```

```
}
```

```
return 0;
```

```
}
```

### Sample code recursive string function

```
void rString(char str[]){
```

```
if(str == NULL) return;
```

```
char *start = str; // pointer of the first char
```

```
char *end = str + strlen(str) - 1; // pointer of the last char
```

```
while(start < end){
```

```
char temp = *start; // swap the value
```

```
*start = *end; // swap the pointer address
```

```
*end = temp
```

```
start++;
```

```
end--;
```

```
}
```

```
}
```

## Functions

Preprocessor

Which is #include <stdio.h>

this allows us to start from c source

code to the preprocessor to the compiler.

Not that 0 is traditionally starts

in column 1 but can be placed lower if needed.

But it must occur before you start any code.

-----Preprocessor directives

Directive Meaning

#include Include the contents of a text file.

#define Define a macro.

#undef Cancel a previous #define.

#if If a test succeeds, take specified action.

#ifdef If a macro is defined, take specified actions.

#ifndef Opposite of #ifdef--if a macro is not defined, take specified actions.

#else If the previous #if, #ifdef, or #ifndef fails, take specified actions.

#endif Mark the end of an #if, #ifdef, or #ifndef body.

#elif "Else if"--a way around nested #if-else constructs.

#line Set line number for the compiler to use when issuing

warning or

error messages.

#error Specify a compile-time error and an accompanying message.

#pragma Provide implementation-specific information to the compiler.

#\_ Ignore this line.

```
#define Normal Meetings= 3;
```

```
#define Special Meetings= 20;
```

```
#define TimePerMeeting= 20;
```

```
#define Total Meetings = NormalMeetings + SpecialMeetings;
```

This macro of totmeetings will expand to 3+5=5.

This is generally not ideal,

because Totalmeeting is kind of consider as a variable.

But something like #define TotalTime = TotalMeetings \* TimePerMeeting;

is equivalent to

```
#define TotalTime = NormalMeetings + SpecialMeetings * TimePerMeeting;
```

This will expand into 3+2\*20 = 43 instead of messing up and equaling to 100.

Note that you can't override a #define function with another define function.

you will have to use the #undef

function to undefine the original define function before redefining it.

Example:

```
#define TRUE 1
```

```
#define TRUE 2
```

This is illegal

but

```
#define TRUE 1
```

```
#undef TRUE
```

```
#define TRUE 2
```

this is legal.

### Swapping Variables

```
void swap(int *a, int *b){
```

```
int temp = *a;
```

```
*a = *b;
```

```
*b = temp;
```

```
}
```

\* symbol is known to be a pointer

declaration or pointer dereferencing.

Void is used to as a return function

without a value, or parameters.

You can't declare a data type as a

void.

< type.h > ← is used for characters. Allowing

characters to convert between uppercase

and lowercase.

< string.h > → allows the implementation of

strings and string modification.